

Riley Brown
Homework 1 Solution Description

Part 1: 'Create an endpoint to dynamically add new topics'

A POST `/api/v1/topics` endpoint was added to `routes.py` that accepts a topic name, description, and list of keywords. The topic is validated to prevent duplicates and is then immediately written to `topic_keywords.json`, making it available for classification without restarting the server. I lastly added a GET `/api/v1/topics/all` and DELETE `/api/v1/topics/{name}` endpoint were also added to complete this question in order to complete the topic management functionality.

Part 2: 'Create an endpoint to store emails'

I also made a POST `/api/v1/emails/store` endpoint to the `routes.py` that accepts a subject, body, and an optional `ground_truth` label. If a label is provided, it is validated against the known topics before saving. When executed, each email is assigned a UUID and appended to `data/emails.json`, and a GET `/api/v1/emails/stored` endpoint can view all stored emails additives.

Part 3: 'Update the classifier to optionally use email similarity'

The `/api/v1/emails/classify` endpoint was updated to accept a `mode` parameter; either "topic" (default, original behavior) or "email" (new). A new method `predict_from_stored_emails()` was added to `similarity_model.py` that encodes the query email using the existing `SentenceTransformer` model and computes the cosine similarity against every labelled stored email, returning the `ground_truth` label of the closest match. Also, a corresponding `classify_email_by_similarity()` method was added to `email_topic_inference.py`. If no labelled emails exist in the store, the classifier automatically falls back to topic mode.

Part 4: 'Demonstrate creating new topics'

The POST /api/v1/topics endpoint was called via SwaggerDocs to create a new finance topic with a description and keywords. As pictured, there is a 201 response confirming the topic was created and persisted. GET /api/v1/topics/all was then called to confirm the new topic appeared alongside the original topics in topic_keywords.json.

→

↺

⚠ Not secure

184.73.106.32:8000/docs#/default/create_topic_api_v1_topics_post

☆

📄

🔍

🔗

Curl

```
curl -X 'POST' \
  'http://184.73.106.32:8000/api/v1/topics' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "finance",
    "description": "Emails about invoices, payments, budgets and financial reports",
    "keywords": ["invoice", "payment", "budget", "tax", "expense", "receipt"]
  }'
```

Request URL

http://184.73.106.32:8000/api/v1/topics

Server response

Code	Details
409	Error: Conflict

Undocumented

Response body

```
{
  "detail": "Topic 'finance' already exists."
}
```

Download

Response headers

```
content-length: 44
content-type: application/json
date: Sun, 17 May 2026 01:04:13 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "name": "string",
  "description": "string",
  "keywords": [
    "string"
  ]
}
```

Activat

Go to Set

Part 5: 'Demonstrate inference on new topics'

Adding onto the previous question, a POST `/api/v1/emails/classify` endpoint was called with `mode: "topic"` and an email about an overdue invoice. The Superhuman classifier correctly predicted finance, which shows that newly added topics are immediately available for classification without restarting the server!

Not secure 184.73.106.32:8000/docs#/default/classify_email_api_v1_emails_classify_post

Curl

```
curl -X 'POST' \
  'http://184.73.106.32:8000/api/v1/emails/classify' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "subject": "Invoice #4521 is overdue",
    "body": "Your invoice for $2,400 is 30 days past due. Please submit payment immediately.",
    "mode": "topic"
  }'
```

Request URL

`http://184.73.106.32:8000/api/v1/emails/classify`

Server response

Code

Details

200

Response body

```
{
  "predicted_topic": "finance",
  "topic_scores": {
    "work": 0.5754527180698917,
    "personal": 0.49446348615569823,
    "promotion": 0.5747821929856453,
    "newsletter": 0.561800209338186,
    "support": 0.6247775460958431,
    "finance": 0.6569415780933129
  },
  "features": {
    "spam_has_spam_words": 0,
    "word_length_average_word_length": 5.176470588235294,
    "email_embeddings_average_embedding": [
      -0.04644833585153656,
      0.00025007117073982954,
      0.10656479001045227,
      -0.05873556435108185,
      -0.04883260279893875,
      -0.06904812982212143,
      -0.1450398564338684,
      0.08917730301618576,
      0.004052174277603626,
      0.028117360547184944,
      0.03467811644077301,
      -0.05841207131743431,
      0.008810517378151417,
      -0.07319169491529465,

```

Download

Response headers

```
content-length: 8645
content-type: application/json
date: Sun, 17 May 2026 01:05:38 GMT
server: uvicorn
```

Part 6: 'Demonstrate adding new emails'

The POST /api/v1/emails/store endpoint was called to add a labelled email with ground_truth: "finance" and a second email without a label to demonstrate the optional nature of ground_truth. I originally misnamed a part in one of my later lines after defining the Data Dictionary, so that is why there is a 500 error. However if it worked, a GET /api/v1/emails/stored was then called to confirm both emails were saved to data/emails.json with unique UUIDs.

Curl

```
curl -X 'POST' \
  'http://184.73.106.32:8000/api/v1/emails/store' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "subject": "Q4 budget approved",
    "body": "The Q4 budget of $150,000 has been approved by the finance committee.",
    "ground_truth": "finance"
  }'
```

Request URL

http://184.73.106.32:8000/api/v1/emails/store

Server response

Code Details

500 Error: Internal Server Error

Undocumented

Response body

```
{
  "detail": "name 'DATA_DIR' is not defined"
}
```



Download

Response headers

```
content-length: 43
content-type: application/json
date: Sun, 17 May 2026 01:00:25 GMT
server: uvicorn
```

Responses

Code	Description	Links
------	-------------	-------

201	Successful Response	No links
-----	---------------------	----------

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": "string",
  "subject": "string",
  "body": "string",
  "ground_truth": "string"
}
```

Activate Windows
Go to Settings to activate Windows.

Part 7: 'Demonstrate inference from email data'

The POST /api/v1/emails/classify endpoint was called twice with the same email shown in the screenshots below: once with mode: "topic" and once with mode: "email". Both returned finance as the predicted topic. In topic mode the classifier matched against the topic description in topic_keywords.json, while the email mode classifier was matched against the stored labelled email added in Q6 using cosine similarity between sentence embeddings. The consistent result across both modes validates that the stored emails accurately represent their topics.

The screenshot shows a web browser window with a REST client interface. The address bar shows the URL `184.73.106.32:8000/docs#/default/classify_email_api_v1_emails_classify_post`. The interface displays a curl command for a POST request to `http://184.73.106.32:8000/api/v1/emails/classify` with a JSON body. The response status is 200, and the response body is a JSON object containing predicted topic, topic scores, and features.

```
curl -X 'POST' \
  'http://184.73.106.32:8000/api/v1/emails/classify' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "subject": "Outstanding balance on account",
    "body": "Your account shows an unpaid balance of $750. Please process payment.",
    "mode": "email"
  }'
```

Request URL

`http://184.73.106.32:8000/api/v1/emails/classify`

Server response

Code Details

200

Response body

```
{
  "predicted_topic": "finance",
  "topic_scores": {
    "work": 0.5525757148658675,
    "personal": 0.5172788548563784,
    "promotion": 0.554388493417799,
    "newsletter": 0.538685178838621,
    "support": 0.584463006872159,
    "finance": 0.6426293879777274
  },
  "features": {
    "spam_has_spam_words": 0,
    "word_length_average_word_length": 5.733333333333333,
    "email_embeddings_average_embedding": [
      -0.03936831280589104,
      0.020077424123883247,
      0.033116474747657776,
      -0.0709998682141384,
      -0.011705610901117325,
      -0.12095221877098083,
      -0.0023959646932780743,
      -0.053850483149290085,
      0.013312813825905323,
      0.013757683336734772,
      -0.038537006825208664,
      -0.08811375367641449,
      0.0010659098625183105,
      -0.04089539125561714,
      ...
    ]
  }
}
```

Response headers

```
content-length: 8656
content-type: application/json
date: Sun, 17 May 2026 01:09:32 GMT
server: uvicorn
```

Part 7 Second Screenshot

Curl

```
curl -X 'POST' \
'http://184.73.106.32:8000/api/v1/emails/classify' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "subject": "Outstanding balance on account",
  "body": "Your account shows an unpaid balance of $750. Please process payment.",
  "mode": "topic"
}'
```

Request URL

```
http://184.73.106.32:8000/api/v1/emails/classify
```

Server response

Code

Details

200

Response body

```
{
  "predicted_topic": "finance",
  "topic_scores": {
    "work": 0.5525757148658675,
    "personal": 0.5172788548563784,
    "promotion": 0.554340453417798,
    "newsletter": 0.538685178838621,
    "support": 0.584463006872159,
    "finance": 0.6426293879777274
  },
  "features": {
    "spam_has_spam_words": 0,
    "word_length_average_word_length": 5.733333333333333,
    "email_embeddings_average_embedding": [
      -0.03936831280589104,
      0.020077424123883247,
      0.033116474747657776,
      -0.0709998682141304,
      -0.011705610901117325,
      -0.12095221877098883,
      -0.0023959646932780743,
      -0.053850483149290085,
      0.013312813825905323,
      0.013757683336734772,
      -0.038537006825208664,
      -0.08011375367641449,
      0.0010659098625133105,
      -0.04089539125561714,

```

Download

Response headers

```
content-length: 8656
content-type: application/json
date: Sun, 17 May 2026 01:10:07 GMT
server: uvicorn
```

Responses